Projet informatique

Gestion des colis et optimisation du chargement d'un camion

Rapport technique

Auteur : [Prénom NOM]

Langage : Python 3

Date : Octobre 2026

Dépôt GitHub : https://github.com/BilalBSK/gestion-colis

Sommaire

1. Introduction et choix du langage
2. Organisation du projet
3. Modélisation des données
4. Gestion des colis : ajout, affichage, recherche
5. Tri des colis
6. Interface utilisateur
7. Passage à l'échelle
8. Optimisation du chargement du camion
9. Prise en compte de la priorité
10. Étude des performances
11. Synthèse des choix algorithmiques majeurs
12. Questions de réflexion
13. Conclusion

1. Introduction et choix du langage

1.1 Contexte

Une entreprise de livraison prépare chaque jour le chargement de ses camions à partir d'une liste de colis. Chaque colis est décrit par un identifiant, un poids, une valeur, une priorité et une destination.

Le programme réalisé doit répondre à deux besoins :

- **gérer les colis** : ajouter, afficher, rechercher et trier une collection pouvant contenir jusqu'à 100 000 colis, voire davantage ;
- **préparer un camion** : choisir les colis à charger sans dépasser la capacité du camion, en transportant la plus grande valeur possible et en favorisant les colis prioritaires.

1.2 Choix du langage : Python

Le projet est réalisé en **Python 3**, avec la bibliothèque standard uniquement. Ce choix repose sur trois raisons :

- **Lisibilité.** Le code Python reste proche du pseudo-code. L'évaluation portant d'abord sur les choix algorithmiques, on peut se concentrer sur les algorithmes plutôt que sur la gestion manuelle de la mémoire (allocation, libération, tailles de tableaux).
- **Outils intégrés.** La bibliothèque standard fournit tout le nécessaire pour tester (`unittest`), mesurer les temps d'exécution (`time`) et la mémoire (`tracemalloc`), et lire des fichiers CSV (`csv`).
- **Structures de données prêtes à l'emploi** (listes dynamiques, dictionnaires), dont le fonctionnement interne et la complexité sont connus et seront expliqués.

Contrepartie assumée : Python est un langage interprété. Une même boucle s'exécute en général 10 à 100 fois plus lentement qu'en C. Cela ne change pas les complexités, mais cela change les temps mesurés. L'étude des performances en tiendra compte.

Respect du sujet (partie 9) : les algorithmes qui constituent le cœur du travail (tris, optimisation du chargement) sont **implémentés à la main**, et leur fonctionnement est expliqué dans ce rapport. Lorsqu'une fonction native est utilisée pour une opération courante, l'algorithme sous-jacent et sa complexité sont précisés. Aucune bibliothèque externe n'est utilisée.

2. Organisation du projet

2.1 Arborescence

Le dépôt respecte l'organisation demandée (dossier `src/`, `README.md`, `rapport.pdf`), complétée par quatre dossiers utiles :

```
projet/
├── src/          code source du programme
├── tests/        tests unitaires
├── donnees/      fichiers de colis (exemple du sujet)
├── docs/         source de ce rapport et captures d'écran
├── outils/       génération du rapport, captures, mesures
├── README.md     présentation et mode d'emploi
└── rapport.pdf   ce rapport
```

2.2 Démarche de travail

Le projet a été réalisé par étapes successives. Chaque étape a été **validée** (tests unitaires et exécution du programme) avant de passer à la suivante, puis enregistrée par un commit sur le dépôt GitHub. L'historique des commits reflète donc la progression du travail.

3. Modélisation des données

3.1 Informations portées par un colis

Le sujet impose au minimum cinq informations par colis. Le tableau suivant précise, pour chacune, le type retenu et les contraintes vérifiées à la création du colis.

Attribut	Type	Contrainte	Justification
identifiant	entier	≥ 0, unique	Sert de clé pour retrouver un colis. L'unicité est vérifiée par la collection (partie 4).
poids	entier (kg)	> 0	Un poids nul ou négatif n'a pas de sens physique. Le choix d'un **entier** est expliqué ci-dessous.
valeur	entier	≥ 0	Une valeur nulle reste possible (colis sans valeur marchande).
priorité	entier	de 1 à 5	Échelle fermée, cohérente avec les exemples du sujet (valeurs de 1 à 5).
destination	texte	non vide	Les espaces en début et en fin sont supprimés (« Paris » saisi avec un espace en trop désigne toujours Paris).

Deux conventions à préciser

- **Sens de la priorité.** Le sujet ne précise pas si 1 ou 5 est la priorité la plus forte. Nous retenons **5 = priorité la plus forte** et 1 = la plus faible, conformément à l'expression « priorité importante » utilisée dans la partie 6 : une valeur élevée correspond à un colis important.
- **Poids entier.** Tous les poids du sujet sont des nombres entiers de kilogrammes. Les conserver sous forme d'entiers est indispensable pour l'algorithme de chargement (partie 8), qui utilise un tableau indexé par la capacité du camion (0, 1, 2, ... kg). Si une précision plus fine était nécessaire, il suffirait d'exprimer les poids en hectogrammes ou en grammes, sans changer l'algorithme.

3.2 Représentation d'un colis en mémoire

Problème	Représenter un colis de façon fiable et compacte, sachant que le programme doit en manipuler jusqu'à 100 000, voire davantage.
Solution retenue	Une **classe de données** Python (`@dataclass`) nommée `Colis`, déclarée avec deux options : • `slots=True` : les cinq attributs sont rangés dans des emplacements fixes de l'objet, au lieu d'un dictionnaire propre à chaque colis ; • `frozen=True` : le colis est **immuable** après sa création.
Complexité en temps	Création et accès à un attribut : O(1).
Complexité en espace	O(1) par colis, soit O(n) pour n colis. Mesure : environ **112 octets par colis**.
Alternatives	Un dictionnaire par colis, une `dataclass` classique (sans `slots`), un tuple nommé (`namedtuple`), ou des tableaux séparés par attribut (un tableau des poids, un tableau des valeurs, etc.).
Justification	**Mémoire :** c'est la représentation la plus compacte parmi celles qui restent lisibles (mesure ci-dessous) : deux fois moins qu'un dictionnaire par colis. **Fiabilité :** l'immuabilité empêche de modifier par erreur l'identifiant d'un colis déjà rangé dans la collection, ce qui rendrait l'index incohérent. **Lisibilité :** on écrit `colis.poids` plutôt que `colis["poids"]` ou `colis[1]`. Les tableaux séparés par attribut seraient encore plus compacts, mais beaucoup moins lisibles. Cette piste n'est intéressante qu'à partir de plusieurs millions de colis (partie 12).

La mesure suivante compare la mémoire nécessaire pour créer 100 000 colis avec chacune des représentations envisagées (script `outils/mesure_memoire_colis.py`, mesure réalisée avec le module `tracemalloc`, liste contenant les colis comprise) :

Représentation	Mémoire pour 100 000 colis	Par colis
Dictionnaire par colis	22,4 Mo	224 octets
`dataclass` sans `slots`	15,2 Mo	152 octets
Tuple nommé (`namedtuple`)	13,6 Mo	136 octets
`dataclass` avec `slots` (retenu)	**11,2 Mo**	**112 octets**

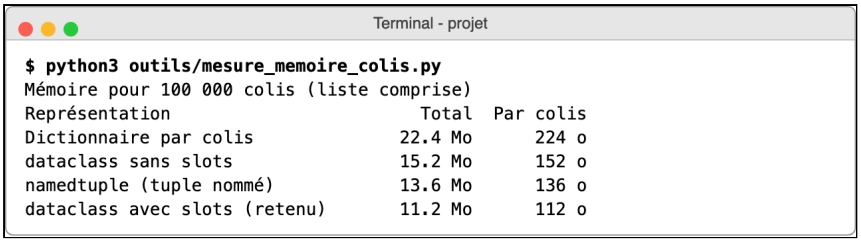


Figure 1 - Mesure de la mémoire occupée selon la représentation choisie

3.3 Validation des données à la création

Chaque colis est vérifié au moment de sa création (méthode `__post_init__`, coût O(1)). Un colis incohérent ne peut donc jamais entrer dans le programme : les algorithmes suivants n'ont pas à se méfier des données. Les règles vérifiées sont :

- identifiant, poids, valeur et priorité sont des **entiers** (un nombre décimal ou un texte est refusé) ;
- poids > 0, valeur ≥ 0, identifiant ≥ 0, priorité comprise entre 1 et 5 ;
- destination non vide.

En cas d'erreur, une exception explicite est levée (par exemple « la priorité doit être comprise entre 1 et 5 »). L'interface pourra ainsi afficher un message clair à l'utilisateur au lieu de s'arrêter.

3.4 Données d'exemple et fichier CSV

Les cinq colis du sujet sont enregistrés dans le fichier `donnees/colis_exemple.csv` :

```
identifiant;poids;valeur;priorite;destination
101;8;20;2;Paris
102;3;10;5;Lille
103;12;40;1;Lyon
104;5;25;4;Paris
105;2;15;3;Lille
```

Le format CSV a été choisi parce qu'il est simple, lisible par un humain et ouvrable dans un tableur. Le séparateur « ; » est celui des tableurs en français, où la virgule sert de séparateur décimal. Le module `stockage.py` lit et écrit ce format, en O(n) en temps et en mémoire pour n colis. Si une ligne est invalide, le message d'erreur indique son **numéro** (par exemple « ligne 3 : le poids doit être strictement positif »), ce qui permet de corriger le fichier facilement.

3.5 Validation de l'étape

Dix-sept tests unitaires (`tests/test_colis.py`) vérifient :

- la création d'un colis valide, y compris aux bornes (priorité 1 et 5, valeur 0) ;
- le refus de chaque donnée invalide : poids nul ou négatif, valeur négative, priorité hors de l'intervalle, identifiant négatif, destination vide, types incorrects ;
- l'immuabilité d'un colis ;
- la lecture exacte des cinq colis du sujet, puis l'écriture et la relecture d'un fichier sans perte ;
- la détection des fichiers mal formés (en-tête incorrect, colonne manquante, valeur non numérique), avec le bon numéro de ligne.

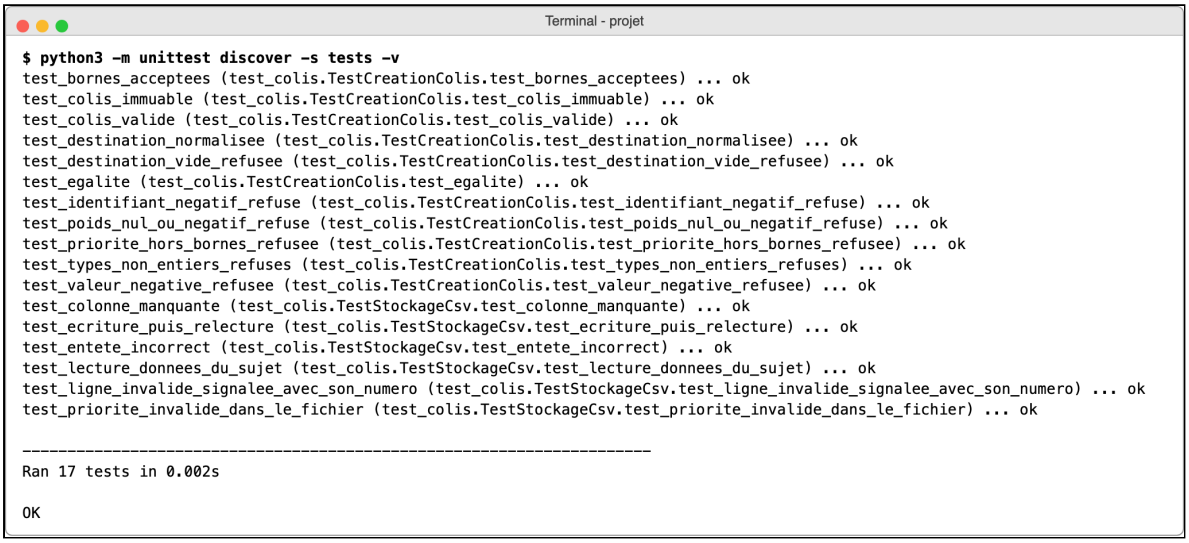


Figure 2 - Exécution des tests unitaires de l'étape 1 : 17 tests réussis